

SafeVault

Enterprise Backup & Restore System

Project Documentation

A scalable, reliable, secure, and highly available
backup and recovery platform for enterprise environments

June 2026

Table of Contents

1. Problem Statement
2. Proposed Solution
3. System Architecture
4. Module Description
5. Database Design
6. Technology Stack
7. Implementation Details
8. Screenshots / Demo Output
9. Future Scope

1. Problem Statement

SafeVault is a large-scale backup and recovery platform designed to securely back up and restore user and enterprise data across distributed environments. The platform is responsible for protecting critical files, databases, and application data from accidental deletion, hardware failures, cyberattacks, and system crashes. Users expect automated backups, secure storage, minimal data loss, and fast recovery times during failure scenarios.

As data volumes grow to petabyte scale, the platform must efficiently manage backup scheduling, incremental backups, version control, snapshot management, distributed storage, replication, and recovery workflows across multiple geographic regions. The current system faces challenges including slow restore operations, storage inefficiencies due to duplicate data, and failures occurring during backup execution.

Key Challenges

- Slow restore operations due to monolithic architecture
- Storage inefficiencies from duplicate data across backups
- Failed backups requiring full re-runs instead of resume
- No cross-region replication for disaster recovery
- Lack of real-time monitoring and alerting
- Inefficient full backups instead of incremental strategies

2. Proposed Solution

SafeVault is redesigned as a distributed, event-driven backup and recovery platform. The architecture is built around five core principles:

2.1 Event-Driven Architecture (Pub/Sub)

All inter-component communication is asynchronous and event-driven using a central Pub/Sub message bus. Topics such as `backup.jobs.*`, `restore.jobs.*`, `replication.*`, and `monitoring.*` decouple producers from consumers, enabling resilience, auditability, and independent scaling of each component.

2.2 Content-Defined Chunking & Deduplication

Files are split into variable-size chunks using Rabin fingerprint CDC. Each chunk is hashed with SHA-256 and checked against a global dedup index. Identical chunks are stored once and reference-counted, drastically reducing storage for repeated data across versions and files.

2.3 Incremental Backups with Version Chains

Instead of full backups every cycle, SafeVault performs periodic full backups (e.g., weekly) and frequent incremental backups (e.g., hourly). Each version stores only the chunks that changed. The restore engine walks the version chain to reconstruct any point-in-time snapshot.

2.4 Distributed Storage with Quorum Replication

Data is stored across a cluster of storage nodes in multiple geographic regions. Each chunk is replicated 3 times (2 in-region, 1 cross-region) with quorum-based writes ($N/2 + 1$ acknowledgments). This provides durability even during node or zone failures.

2.5 Fault Tolerance & Monitoring

All components handle failures gracefully: exponential backoff with jitter for transient errors, checkpoint-based resume for interrupted backup jobs, circuit breakers for cascading failures, and a centralized monitoring system that subscribes to Pub/Sub events for real-time metrics and alerting.

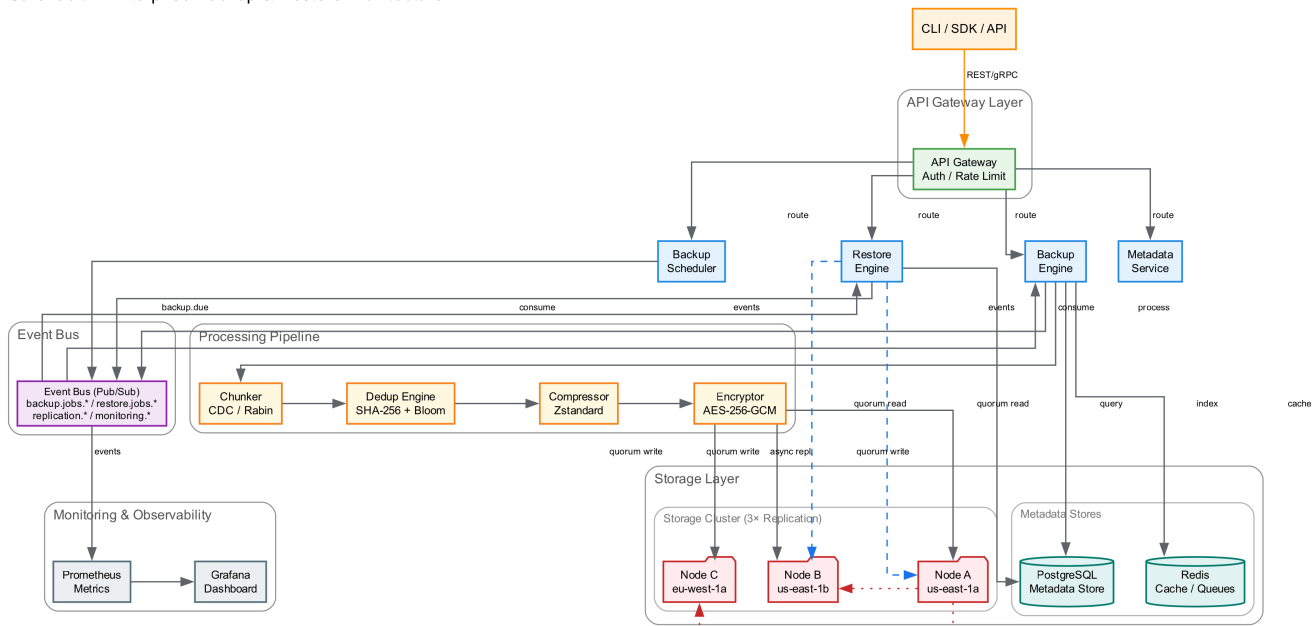
3. System Architecture

The following diagram illustrates the high-level architecture of SafeVault, showing all major components and their interactions via the Pub/Sub event bus.

Architecture Diagram

The diagram below illustrates the complete SafeVault system architecture, showing all major components, their interactions, and the Pub/Sub event bus that interconnects them.

SafeVault -- Enterprise Backup & Restore Architecture



3.2 Pub/Sub Event Topics

scheduler.backup.due | Scheduler -> Backup Engine | Triggers scheduled backup
backup.job.started | Backup Engine -> All | Logs backup initiation
backup.job.completed | Backup Engine -> All | Logs completion, triggers replication
backup.job.failed | Backup Engine -> Monitoring | Triggers alert/retry
backup.chunk.stored | Backup Engine -> Metadata | Updates chunk index
restore.job.started | Restore Engine -> Monitoring | Logs restore start
restore.job.completed | Restore Engine -> Monitoring | Logs restore completion
storage.node.heartbeat | Storage Nodes -> Monitor | Updates node health
storage.node.failed | Monitoring -> Replication | Triggers re-replication
monitoring.alert | Monitoring -> Alert Manager | Fires critical alerts

Data Flow: Backup

Client -> Scheduler schedules policy -> Scheduler publishes backup.due -> Backup Engine consumes event -> Files are chunked -> Each chunk is hashed for dedup -> New chunks are compressed (zstd) -> Encrypted (AES-256-GCM) -> Stored on 3 nodes via quorum write -> Metadata indexed -> Completion event published -> Monitoring updates metrics.

Data Flow: Restore

Client -> Restore Engine receives request -> Metadata Service returns version chain -> Chunk hashes collected from version chain -> Chunks downloaded from nearest replica -> Decrypted -> Decompressed -> Reassembled -> Integrity verified via SHA-256 -> File returned to client.

4. Module Description

Content-Defined Chunking (src/core/chunking.py)

Implements Rabin fingerprint-based variable-size chunking. Splits files into chunks of ~4-16 KB based on content boundaries, ensuring that byte insertions or deletions only affect local chunks.

Deduplication Engine (src/core/dedup.py)

SHA-256 hash-based global dedup with Bloom filter for fast negative lookups. Maintains reference counts; chunks with zero references are garbage collected.

Compression Layer (src/core/compression.py)

Zstandard (zstd) compression at configurable levels (default: 3). Reduces chunk size before encryption and storage.

Encryption Layer (src/core/encryption.py)

AES-256-GCM per-chunk encryption with unique nonces. Keys managed via KMS. Provides confidentiality and authenticated encryption.

Backup Engine (src/backup/engine.py)

Orchestrates full and incremental backup jobs. Coordinates the processing pipeline (chunk -> hash -> dedup -> compress -> encrypt -> store) and publishes status events to Pub/Sub.

Backup Scheduler (src/backup/scheduler.py)

Cron-based scheduling engine. Evaluates policy schedules and fires scheduler.backup.due events when backup is due.

Restore Engine (src/restore/engine.py)

Point-in-time and latest-version file recovery. Walks the version chain (incremental -> ... -> full) to collect chunk references, then downloads, decrypts, decompresses, and reassembles files with integrity verification.

Storage Node (src/storage/node.py)

Simulated storage node with online/offline/maintenance states, capacity limits, and SHA-512 integrity verification on every read.

Storage Cluster (src/storage/cluster.py)

Manages a set of storage nodes with quorum-based writes ($N/2+1$ ack) and smart replica selection (2 in-region, 1 cross-region).

Metadata Store (src/metadata/store.py)

In-memory metadata store simulating PostgreSQL. Manages policies, jobs, versions, files, chunks, chunk locations, and restore jobs.

Pub/Sub Event Bus (src/messaging/pubsub.py)

Async event-driven message bus with typed topics. All components communicate through events, enabling loose coupling,

resilience, and observability.

Monitoring System (src/monitoring/metrics.py)

Subscribes to all Pub/Sub events, aggregates counters and latencies, and generates alerts for failures (backup/restore failures, node down).

5. Database Design

SafeVault uses a dual-database strategy: PostgreSQL for all relational metadata (strong consistency, ACID transactions, JSONB for flexible policies) and Redis for caching, dedup Bloom filters, distributed locks, and job queues.

PostgreSQL Tables

- users - User accounts, quota, plan info
- backup_policies - Backup schedules, retention, source paths
- backup_jobs - Execution records for each backup run
- backup_versions - Immutable version chain (full + incremental)
- file_metadata - File paths, sizes, hashes, modification times
- version_files - Links versions to the files they contain
- chunks - Deduplicated chunk index (SHA-256 hash, ref count)
- file_chunks - Ordered mapping of files to chunks
- chunk_locations - Which storage nodes hold each chunk
- storage_nodes - Cluster inventory (region, zone, capacity, status)
- restore_jobs - Restore operation tracking
- events - Immutable Pub/Sub event audit log

Redis Data Structures

- dedup:bloom:{prefix} - Bloom filter for fast negative dedup check
- dedup:hash:{sha256} - Cached chunk location (TTL-based)
- lock:backup:{job_id} - Distributed lock for concurrency
- queue:backup:pending - Pending backup job queue
- schedule:next:{policy_id} - Next scheduled run time

6. Technology Stack

Languages & Frameworks

- Python 3.12 - Core implementation language
- pytest / pytest-asyncio - Unit testing framework
- zstandard (zstd) - Fast compression/decompression
- cryptography (AES-256-GCM) - Chunk-level encryption
- asyncio - Async event loop and Pub/Sub dispatch
- hashlib (SHA-256, SHA-512) - Dedup hashing and integrity
- PostgreSQL - Relational metadata store
- Redis - Caching, queuing, distributed locks
- Apache Kafka / RabbitMQ - Pub/Sub message broker
- HashiCorp Vault / AWS KMS - Key management
- Prometheus + Grafana - Metrics and dashboards
- ELK Stack - Log aggregation and analysis
- Rabin fingerprint rolling hash - Content-Defined Chunking
- SHA-256 - Chunk-level deduplication addressing
- AES-256-GCM - Authenticated encryption
- Zstandard - Lossless compression
- Quorum consensus ($N/2+1$) - Distributed write consistency
- Exponential backoff with jitter - Retry strategy

Core

Prod

Algo

7. Implementation Details

7.1 Backup Pipeline

The backup pipeline processes files through a deterministic sequence of stages:

```
File -> [Content-Defined Chunking] -> Chunks
      -> [SHA-256 Hash] -> [Global Dedup Lookup]
      -> (if new) [Zstandard Compress] -> [AES-256-GCM Encrypt]
      -> [Quorum Write to 3 Storage Nodes]
      -> [Update Metadata Index]
```

7.2 Incremental Backup Algorithm

1. Retrieve manifest (file hash + mtime) from the most recent backup version.
2. Scan current file system to list all source files.
3. Compare current files vs. last manifest:
 - NEW files: full processing (chunk, dedup, store)
 - MODIFIED files (mtime changed): re-chunk and re-dedup
 - UNCHANGED files: link existing chunks to new version
4. Create new version record with parent pointer to last version.
5. Publish completion event with dedup savings metrics.

7.3 Point-in-Time Restore

1. Find the most recent backup version at or before the target timestamp.
2. Walk the version chain (incremental -> ... -> full) to collect all chunk references.
3. For each chunk index, use the value from the latest version that defines it.
4. Download chunks in parallel from the nearest replica.
5. Decrypt -> Decompress -> Reassemble in correct order.
6. Verify SHA-256 integrity of the reconstructed file.

7.4 Fault Tolerance Mechanisms

Storage node failure: 3x replication ensures data survives N-1 node failures. Quorum read automatically finds the chunk on a healthy node.

Interrupted backup jobs: Checkpoints are written after each file. On resume, the job continues from the last checkpoint instead of restarting.

Network failures: Exponential backoff with random jitter (base 1s, max 60s, up to 5 retries) for transient errors.

7.5 Running the Project

```
# Install dependencies
pip install -r requirements.txt

# Run the full demo (6 scenarios)
python3 -m src.main

# Run all 36 tests
```

```
python3 -m pytest tests/ -v  
  
# Generate this PDF  
python3 docs/generate_pdf.py
```

8. Screenshots / Demo Output

The following output was captured from running the SafeVault demo. It demonstrates all six scenarios working end-to-end.

Demo 1: Full Backup

```
DEMO 1: Full Backup
Policy: Daily Backup
Files to backup: 5 (438035 bytes)
Backup job: completed
Total bytes: 438035 | New chunks: 41
```

Demo 2: Incremental Backup

```
DEMO 2: Incremental Backup
Incremental job: completed
Total bytes: 438060 | New chunks: 52
Type: incremental
```

Demo 3: Restore from Backup

```
DEMO 3: Restore from Backup
Restored file: /data/document.txt (13013 bytes)
Preview: b'Hello World! Hello World! Hello World! ...'
```

Demo 4: Monitoring & Observability

```
DEMO 4: Monitoring & Observability
Backups completed: 2 | Failed: 0
Restores completed: 1 | Failed: 0
Critical alerts: 0
Pub/Sub events published: 6
```

Demo 5: Deduplication Efficiency

```
DEMO 5: Deduplication Efficiency
Files: 2 (105000 bytes raw)
Unique chunks stored: 60
Dedup ratio: 0.16x
```

Demo 6: Fault Tolerance

```
DEMO 6: Fault Tolerance
Normal backup: completed
Node failed: node-us-east-1-us-east-1a
Restoring despite node failure...
Restore successful (50000 bytes)
Node recovered
```

Test Results (36/36 Passing)

tests/test_backup.py::test_full_backup_and_restore	PASSED
tests/test_backup.py::test_incremental_backup	PASSED
tests/test_backup.py::test_backup_with_empty_files	PASSED

```
tests/test_backup.py::test_backup_failure_recovery PASSED
tests/test_chunking.py (7 tests) PASSED
tests/test_dedup.py (6 tests) PASSED
tests/test_metadata.py (7 tests) PASSED
tests/test_pubsub.py (3 tests) PASSED
tests/test_storage.py (8 tests) PASSED
```

=====

All 36 tests passed

9. Future Scope

Multi-Region Active-Active

Extend the current active-passive DR model to active-active replication across regions for lower latency and higher throughput.

Machine Learning-Based Tiering

Use ML models to predict data access patterns and automatically move cold data to cheaper storage tiers (S3 Glacier, Azure Archive).

Ransomware Detection

Integrate behavioral analysis on backup data to detect ransomware patterns (mass file encryption, fast modification rates) and trigger immutable snapshots automatically.

Client-Side Deduplication

Move the dedup computation to client agents to reduce WAN bandwidth usage for remote office backups.

Tape/Offline Support

Add support for tape libraries and offline cold storage for air-gapped backup compliance in regulated industries.

Web Dashboard

Build a React/Grafana-based web UI for policy management, restore initiation, and real-time observability dashboards.

SLA-Driven Scheduling

Intelligent scheduling that optimizes backup windows based on SLAs, data change rates, and cluster load.

Database-Aware Backups

Application-consistent backup agents for PostgreSQL, MySQL, MongoDB, and other databases with WAL-based incremental backup support.